Pré-Processamento de Textos com a ferramenta NLTK

<u>Aula 10</u> – Introdução ao Processamento de Linguagem Natural (PLN)

Eduardo Corrêa Gonçalves

02/04/2026

Sumário

Introdução

Conceitos Preliminares

- Corpus, Corpora e Corpus Anotado
- Palavra
- Token, Type e Vocabulário
- Normalização

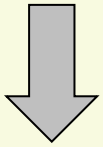
NLTK

- Tokenização
- Extraindo Frequências
- Bigramas, Trigramas e *n*-Gramas
- Removendo Stop Words
- Stemming

Introdução (1/3)

- **O que é Pré-Processamento de Texto?**
  - Conjunto de tarefas que tem por objetivo...
  - ... realizar a **conversão** de palavras, frases ou até mesmo caracteres do texto de entrada para um **formato** que seja **mais conveniente** para os algoritmos lidarem com ele (FINATTO et al., 2024; JURAFSKY e MARTIN, 2026).

"Vovó viu a uva"



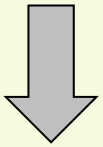
["vovó", "ver", "uva"]

  - O pré-processamento do texto costuma ser **necessário** para o desenvolvimento de **qualquer aplicação** de **PLN**.

Introdução (2/3)

- **Quais as tarefas de pré-processamento?**
  - A mais usual é a **tokenização**.
    - Separação das palavras, números e pontuações do texto.
  - Outra tarefa muito comum é a **normalização**.
    - Conversão das palavras para alguma forma padrão.

"Vovó viu a uva"



["vovó", "ver", "uva"]

- **No exemplo acima**:
  - as palavras foram separadas e convertidas p/ minúsculo;
  - a palavra “a” (artigo) foi removida;
  - “viu” (pretérito) foi trocado por “ver” (infinitivo).

Introdução (3/3)

- **Aula de Hoje**
  - A aula de hoje cobre o pré-processamento na prática com o pacote **NLTK** (https://www.nltk.org/index.html).
    - ***Toolkit*** (biblioteca de ferramentas) para a utilização dos princípios de PLN no Python
  - Porém... antes de examinarmos a NLTK, vamos apresentar algumas definições importantes:
    - Corpus
    - Corpora
    - Corpus Anotado
    - Palavra
    - Token
    - Type
    - Vocabulário
    - Normalização de tokens

Link para notebooks com o exemplos da aula:

- Recursos da biblioteca **NLTK** para: Tokenização, Frequência de Palavras e *n*-gramas de palavras, Stop Words e Stemming
- https://colab.research.google.com/drive/1H2izzt2Low-iGfgBqumycmgV4lDYkkxa?usp=sharing

- Os exemplos são baseados nos apresentados em PARDO; SANTOS; SOUZA (2021).

Conceitos Preliminares (1/11)

- **Corpus** (**plural:** Corpora)
  - Conjunto de textos coletados e organizados com algum propósito.
    - Pode ser entendido como um *dataset* em que cada observação corresponde a um documento (texto).

- **Exemplo**: Brown Corpus

- Famoso corpus que contém 500 textos em Inglês.
  - Os textos são de diferentes gêneros: jornalísticos, ficção, acadêmicos etc.
  - Possui ≈ 1 mi de palavras no total.

```
F21 1500 disaster. Superstition has often blended with fact to color reports.
F21 1510    We have learned from earthquakes much of what we now know
F21 1520 about the earth's interior, for they send waves through the earth
F21 1530 which emerge with information about the materials through which they
F21 1540 have traveled. These waves have shown that 1,800 miles below the surface
F21 1550 a liquid core begins, and that it, in turn, has a solid inner core.
F21 1560    Earthquakes originate as far as 400 miles below the surface,
F21 1570 but they do not occur at greater depths. Two unsolved mysteries are
F21 1580 based on these facts. (1) As far down as 400 miles below the surface
F21 1590 the material should be hot enough to be plastic and adjust itself to
F21 1600 twisting forces by sluggish flow rather than by breaking, as rigid surface
F21 1610 rocks do. (2) If earthquakes do occur at such depths, why not deeper?
F21 1620    Knowledge gained from studying earthquake waves has been
F21 1630 applied in various fields. In the search for oil and gas, we make
F21 1640 similar waves under controlled conditions with dynamite and learn from
F21 1650 them where there are buried rock structures favorable to the accumulation
F21 1660 of these resources. We have also developed techniques for recognizing
F21 1670 and locating underground nuclear tests through the waves in the
F21 1680 ground which they generate.   The following discussion of this subject
F21 1690 has been adapted from the book {Causes of Catastrophe} by
F21 1700 L& Don Leet. #THE RESTLESS EARTH AND ITS INTERIOR# <At twelve
F21 1710 minutes after five on the morning of Wednesday>, April 18, 1906,
F21 1720 San Francisco was shaken by a severe earthquake. A sharp tremor
F21 1730 was followed by a jerky roll.
F22 0010    {IN} Ireland's County Limerick, near the River
F22 0020 Shannon, there is a quiet little suburb by the name of Garryowen, which
F22 0030 means "Garden of Owen". Undoubtedly none of the residents realize
F22 0040 the influence their town has had on American military history,
F22 0050 or the deeds of valor that have been done in its name. The cry "Garryowen"!
F22 0060 bursting from the lips of a charging cavalry trooper was
F22 0070 the last sound heard on this earth by untold numbers of Cheyennes,
F22 0080 Sioux and Apaches, Mexican <banditos> under Pancho Villa, Japanese
F22 0090 in the South Pacific, and Chinese and North Korean Communists
F22 0100 in Korea. Garryowen is the battle cry of the 7th U& S& Cavalry
F22 0110 Regiment, "The Fighting Seventh".    Today a battle
```

Conceitos Preliminares (2/11)

- **Corpus Anotado**
  - É um corpus em que cada documento está anotado – ou classificado, ou rotulado, ou etiquetado de alguma forma (FREITAS 2024).
    - A anotação pode ser de diferentes naturezas e pode estar associada a diferentes segmentos de texto (palavras, expressões, frases, parágrafos ou o texto inteiro).

  - **Exemplos**:
    - Indicar se uma postagem em rede social expressa discurso de ódio;
    - Indicar qual sentimento um texto ou palavra evoca.
    - Indicar a classe gramatical de uma palavra;
    - Indicar se duas frases são sinônimas ou não;
    - Relacionar uma palavra a pronomes e todas as outras maneiras pelas quais esta palavra é referida ao longo de um texto;
    - Indicar se uma frase é uma boa legenda para uma imagem;
    - ...

Conceitos Preliminares (3/11)

- **Corpus Anotado – Exemplo**: **sentiment140**
  - Corpus anotado c/ o sentimento (positivo ou negativo) de 359 tweets.

sentiment140.csv

Preview | Code | Blame 360 lines (360 loc) · 33.4 KB Code 55% faster with GitHub Copilot

Search this file

1	sentiment	review
2	positive	@stellargirl I loooooooovvvvvveee my Kindle2. Not that the DX is cool, but the 2 is fantastic in its own right.
3	positive	Reading my kindle2... Love it... Lee childs is good read.
4	positive	Ok, first assesment of the #kindle2 ...it fucking rocks!!!
5	positive	@kenburbary You'll love your Kindle2. I've had mine for a few months and never looked back. The new big one is huge! No need for remorse! :)
6	positive	@mikefish Fair enough. But i have the Kindle2 and I think it's perfect :)
7	positive	@richardebaker no. it is too big. I'm quite happy with the Kindle2.
8	negative	Fuck this economy. I hate aig and their non loan given asses.
9	positive	Jquery is my new best friend.
10	positive	Loves twitter
11	positive	how can you not love Obama? he makes jokes about himself.
12	negative	@Karoli I firmly believe that Obama/Pelosi have ZERO desire to be civil. It's a charade and a slogan, but they want to destroy conservatism
13	positive	House Correspondents dinner was last night whoopi, barbara & sherri went. Obama got a standing ovation

Conceitos Preliminares (4/11)

- **Palavra**
  - De um modo geral, os modelos de PLN trabalham as **palavras** como **unidade primária** de processamento.
  - Nas **línguas europeias** (como Português), podemos considerar que uma palavra é uma unidade grafológica delimitada entre espaços em branco na representação gráfica, ou entre um espaço em branco e um sinal de pontuação (FINATTO et al., 2024).
    - Por outro lado, alguns idiomas, como Japonês, não usam espaços entre as palavras!!!
    - Além disso, mesmo em textos de línguas europeias há **exceções** para a definição acima (veremos...)

Conceitos Preliminares (5/11)

- **Nos textos, as palavras *quase* sempre:**
  - Estão separadas por um **espaço em branco** ou entre **espaço em branco e sinal de pontuação**.
  - Possuem apenas **letras**, **números** e **hífen**.
    - "Vovó viu a uva.",
    - "Cadê meu guarda-chuva?",
    - "2024 é um ano bissexto.",
    - "Ih!"
    - "pneumoultramicroscopicossilicovulcanoconiótico"*

* Essa é a maior palavra da língua portuguesa: https://www.bbc.com/portuguese/curiosidades-43938059

Conceitos Preliminares (6/11)

- **Porém, há (um bom número de) exceções... (1/2)**
  - Veja a frase abaixo:
    - "Ana Paula comprou 2 livros em Porto Alegre."
  - São **8 palavras** considerando a definição do slide anterior.
    - ... Mas não seriam 6 palavras se considerarmos que "Ana Paula" e "Porto Alegre" são palavras compostas?
    - ... E será que "2" é realmente uma palavra?
      - A resposta é: **depende** da finalidade da **aplicação**!

Conceitos Preliminares (7/11)

- **Porém, há (um bom número de) exceções... (2/2)**
  - De fato, a definição "fura" também para o tratamento de abreviaturas, siglas, "internetês" e outras situações.

palavras compostas	"Rio de Janeiro", "rock 'n' roll" (contêm, espaços, mas não deveriam ser separadas).
hashtags	"#riodejaneiro", "#verao", "#samba"
emoticons	"eu <3 #vilavelha", "você vai rir muito :)"
usernames	"@jgoncalves, poderia ver isso?"
abrev. do "internetês"[1]	"isso foi d+" (demais)
abrev. convencionais[2]	"s.d." (sem data), "a/c" (aos cuidados de)
URLs, e-mails	"https://brasileiraspln.com/livro-pln/"
preços, porcentagens, valores etc.	"reajuste de 9%", "R$ 1,99", "orçamento de 5 bi".
apóstrofo[1]	"cobra-d'água" (não deve ser separada)
apóstrofo[2]	"I'm" (não tem espaço, mas são 2 palavras: "I" e "am")

  - Dependendo da aplicação, tratar alguns casos acima pode ser ou não relevante.
  - **Exemplo**: na análise de sentimentos, é comum remover *usernames*

Conceitos Preliminares (8/11)

- **Tokens, Types e Vocabulário (V)**
  - Em um **texto** *T*, as **palavras** e **sinais de pontuação** são considerados **tokens**.
    - O número total de tokens é representado por *N*.
  - Os **tokens únicos** do texto são chamados de **types**.
    - O conjunto de types é chamado de **Vocabulário** (*V*).
  - **Exemplo**: "Eu vou. Por que não? Por que não?"
    - 11 tokens (*N* = 11): [“Eu”, “vou”, “.”, “Por”, “que”, “não”, “?”, “Por”, “que”, “não”, “?”]
    - 7 types (|*V*| = 7): *V* = {“Eu”, “vou”, “.”, “Por”, “que”, “não”, “?”}

Conceitos Preliminares (9/11)

- **Tokens, Types e Vocabulário (V)**
  - Na tabela abaixo, o número de types e tokens de diversos corpora conhecidos.
  - **IMPORTANTE**: MAC-MORPHO e Floresta são corpora anotados da língua portuguesa, ambos disponíveis na NLTK.

Corpus	|V| (num. de types)	N (num. de tokens)
Shakespeare	31 mil	884 mil
Brown	28 mil	1 milhão
Switchboard telefone conversations	20 mil	2,4 milhões
COCA	2 milhões	440 milhões
Google n-grams	13 milhões	1 trilhão
MAC-MORPHO	67,9 mil	1,17 milhões
Floresta	29,4 mil	211,8 mil

Conceitos Preliminares (10/11)

- **Riqueza Léxica**
  - Além da quantidade de types e tokens, a **riqueza léxica** é uma medida que pode ajudar a medir a complexidade de um corpus.
    - Divisão da quant. de types pela quant. de tokens → $RL = |V| \div N$
  - Varia entre 0 e 1. Quanto maior, mais "rico" o texto (e potencialmente mais difícil de se processar).
  - **Exemplos**:
    - "Todos os meus conhecidos têm sido campeões em tudo."
    - RL = 10 / 10 = <u>1,00</u>
    - "que não é o que não pode ser que não é o que não pode ser que não é o que não pode ser que não é o que não pode ser que não é"
    - RL = 6 / 35 = <u>0,17</u>

Conceitos Preliminares (11/11)

- **Normalização**
  - Tarefa que consiste em converter as palavras para alguma forma padrão (FINATTO et al., 2024).
  - **Exemplos**:
    - Conversão para caracteres minúsculos ("CANTEI" → "cantei").
    - Conversão de abreviações: ("d+" → "demais")
    - Concatenação de expressões multipalavra ("Rio Paraíba" → "rioparaíba").
    - Lematização ("cantarei" → "cantar").
    - Radicalização ("cantarei" → "cant").
  - Os slides seguintes abordarão a tokenização e normalização na ferramenta **NLTK**.

NLTK – Instalação (1/5)

- **Etapas**
  - A NLTK está **disponível no Google Colab**.
  - Caso **deseje instalar** no ambiente Python de **seu microcomputador**, você precisará executar 3 etapas (que felizmente são simples):
    - **Instalar** o pacote
    - **Testar** se a instalação deu certo
    - Fazer o **download dos dados** necessários para o funcionamento.
    - As informações nos próximos slides foram obtidas em https://www.nltk.org/install.html.

NLTK – Instalação (2/5)

- **<u>Etapa 1</u>: Instalação do Pacote**
  - Você vai acessar o prompt de comando
  - E fará a instalação do pacote propriamente dito com o aplicativo pip, digitando: pip install -U nltk.

```
Prompt de Comando - pip in:

Microsoft Windows [versão 10.0.26100.3323]
(c) Microsoft Corporation. Todos os direitos reservados.

C:\Users\ecgrj>pip install -U nltk
Collecting nltk
  Downloading nltk-3.9.1-py3-none-any.whl.metadata (2.9 kB)
Requirement already satisfied: click in c:\python313\lib\site-packages (from nltk) (8.1.8)
Requirement already satisfied: joblib in c:\python313\lib\site-packages (from nltk) (1.4.2)
Collecting regex>=2021.8.3 (from nltk)
  Downloading regex-2024.11.6-cp313-cp313-win_amd64.whl.metadata (41 kB)
Collecting tqdm (from nltk)
  Downloading tqdm-4.67.1-py3-none-any.whl.metadata (57 kB)
Requirement already satisfied: colorama in c:\python313\lib\site-packages (from click->nltk) (0.4.6)
Downloading nltk-3.9.1-py3-none-any.whl (1.5 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.5/1.5 MB 1.9 MB/s eta 0:00:00
Downloading regex-2024.11.6-cp313-cp313-win_amd64.whl (273 kB)
Downloading tqdm-4.67.1-py3-none-any.whl (78 kB)
Installing collected packages: tqdm, regex, nltk
```

NLTK – Instalação (3/5)

- **<u>Etapa 2</u>: Testando se a instalação deu certo**
  - Faça da seguinte forma:
    - Ainda no prompt de comando, digite python, para entrar no aplicativo de linha de comando do python
    - Então teste com o comando import nltk
      - A biblioteca NLTK é grande, por isso a importação leva alguns segundos até ser finalizada (você verá que o prompt ficará travado por um tempinho).
      - Se nenhuma mensagem de erro ocorrer é porque a instalação foi bem-sucedida.

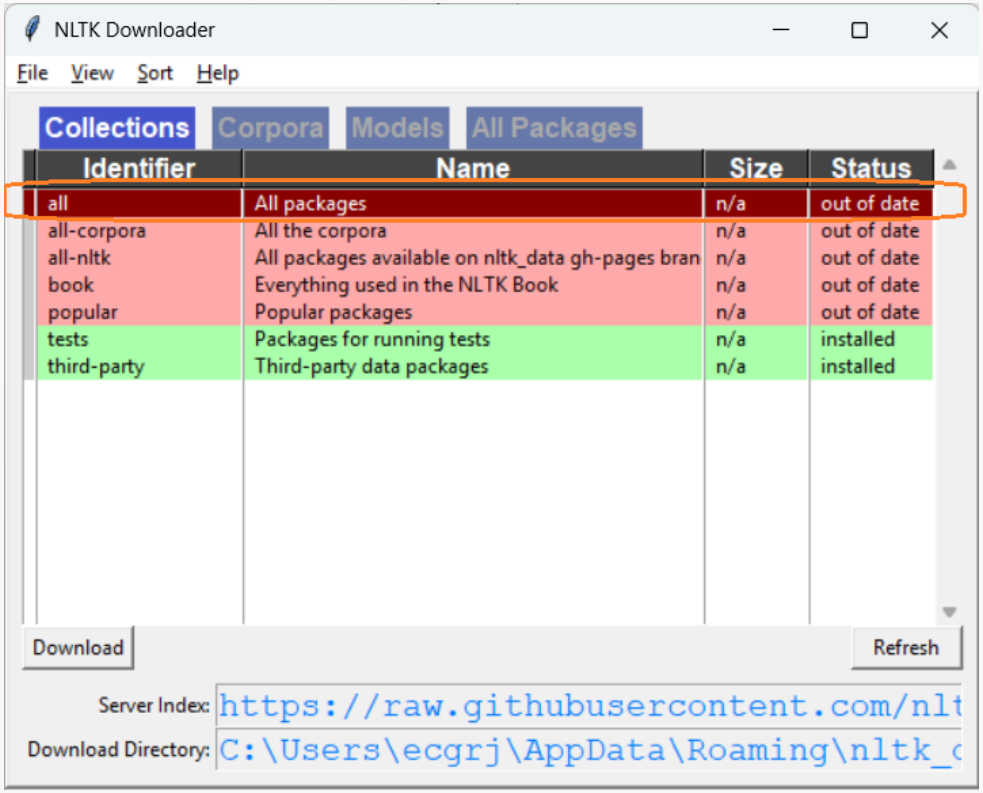
```
C:\Users\ecgrj>python
Python 3.13.2 (tags/v3.13.2:4f8bb39, Feb  4 2025, 15:23:48) [MSC v.1942 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> import nltk
>>> |
```

NLTK – Instalação (4/5)

- **<u>Etapa 3</u>: Fazendo o download dos dados**
  - Significa baixar p/ seu computador vários corpora, léxicos e outros recursos necessários para o funcionamento da NLTK.
  - Ainda **dentro do Python**, logo após fazer o import nltk, você vai digitar: nltk.download()

```
>>> import nltk
>>> nltk.download()
showing info https://raw.githubusercontent.com/nltk/nltk_data/gh-pages/index.xml
```

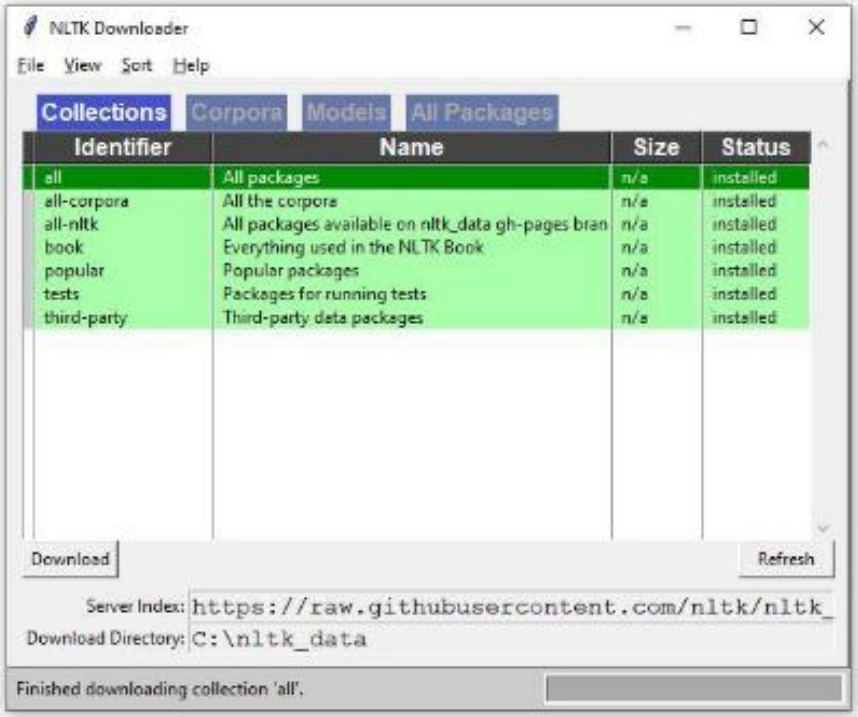
- Na janela que abrirá, selecione **all** e **All packages** (na primeira linha), para instalar todos os pacotes (provavelmente já estará selecionado).
- Depois clique no botão **Download**, no canto inferior esquerdo.



NLTK – Instalação (5/5)

- **<u>Etapa 3</u>: Fazendo o download dos dados**
  - O processo pode demorar mais de 2 minutos.
  - Quando estiver tudo certo, todos os pacotes estarão com a cor verde. Aí você já poderá sair do python se quiser.

```
>>> import nltk
>>> nltk.download()
showing info https://raw.githubusercontent.com/nltk/nltk_data/gh-pages/index.xml
```



Pré-processamento básico com a NLTK

- **Utilização básica**
  - Agora vamos apresentar diversos exemplos de código que demonstram a utilização básica da NLTK.
  - Os exemplos são baseados nos apresentados em PARDO; SANTOS; SOUZA (2021) e estão disponíveis no notebook que já havíamos indicado em um dos slides iniciais da aula:
    - https://colab.research.google.com/drive/1H2izzt2Low-iGfgBqumycmgV4lDYkkxa?usp=sharing
  - Dividimos em 4 partes:
    - Tokenização
    - Frequências
    - *n*-gramas de palavras
    - Normalização: Stop Words e Stemming

Tokenização com a NLTK (1/7)

- **Importação**
  - Primeiro é preciso importar a nltk.
  - No Colab, você precisará ainda fazer o download do módulo **punkt_tab**. necessários:
    - **punkt_tab**: tokenizador que entende algumas abreviações (ex.: Mr.) e colocações.

```
# importa a nltk; e
# realiza o download do módulo necessário para tokenização
# (no colab tem que fazer sempre no início)
# (no Python da sua máquina, só ao instalar a nltk)
import nltk

#nltk.download('punkt')
nltk.download('punkt_tab')
```

- *Se você tiver algum problema use o 'punkt' (versão anterior) em vez do 'punkt_tab'. O punkt aparece em muitos exemplos na Internet, mas foi descontinuado de acordo com →* https://github.com/nltk/nltk/issues/3293

Tokenização com a NLTK (2/7)

- **Tokenização de um documento**
  - Dado um documento *d*, basta usar a função `nltk.word_tokenize(d)` para obter seus tokens.
    - Por padrão, usa versão modificada do *TreebankWordTokenizer* – um tokenizador conhecido, baseado em expressões regulares.
    - O parâmetro `language= 'portuguese'` foi usado para que o tokenizador utilize os sinais de pontuação adequados para o Português
      - *não tenho certeza de que realmente tenha alguma efetividade*

```
d = '''HOJE TEM FEIRA NA PRAÇA DA LIBERDADE! A batata-doce
custa R$ 9,50. Quem vai à feira comprar?'''

tokens = nltk.word_tokenize(d, language='portuguese')

print(tokens)
```

```
> ['HOJE', 'TEM', 'FEIRA', 'NA', 'PRAÇA', 'DA', 'LIBERDADE',
'!', 'A', 'batata-doce', 'custa', 'R', '$', '9,50', '.',
'Quem', 'vai', 'à', 'feira', 'comprar', '?']
```

Tokenização com a NLTK (3/7)

- **Tokenização de um documento**
  - A tokenização de R$ 9,50 não ficou ideal ☹

```
d = '''HOJE TEM FEIRA NA PRAÇA DA LIBERDADE! A batata-doce
custa R$ 9,50. Quem vai à feira comprar?'''

tokens = nltk.word_tokenize(d, language='portuguese')

print(tokens)
```

```
> ['HOJE', 'TEM', 'FEIRA', 'NA', 'PRAÇA', 'DA', 'LIBERDADE',
'!', 'A', 'batata-doce', 'custa', 'R', '$', '9,50', '.',
'Quem', 'vai', 'à', 'feira', 'comprar', '?']
```

Tokenização com a NLTK (4/7)

- **Tokenização de um documento**
  - Podemos normalizar o texto para minúsculo usando aplicando o método lower() na string.
  - É uma técnica de normalização... Assim “FEIRA” e “feira” não vão virar 2 types diferentes !!!
  - *Veja que o tokenizador se confundiu ainda mais no “9,50.”*

```
d = '''HOJE TEM FEIRA NA PRAÇA DA LIBERDADE! A batata-doce
custa R$ 9,50. Quem vai à feira comprar?'''

tokens = nltk.word_tokenize(d.lower(), language='portuguese')

print(tokens)
```

```
> ['hoje', 'tem', 'feira', 'na', 'praça', 'da', 'liberdade',
'!', 'a', 'batata-doce', 'custa', 'r', '$', '9,50.', 'quem',
'vai', 'à', 'feira', 'comprar', '?']
```

Tokenização com a NLTK (5/7)

- **Tokenização de todos os textos de um corpus**
  - Em um corpus estruturado em um DataFrame, basta usar o apply...
  - ... Para criar uma nova coluna com todos os textos tokenizados

```
import pandas as pd

# corpus original
corpus_dantestocks = {'tweet': [
"No momento PETR4 respeita o suporte de R$15,42.",
"28/04 O Ibovespa fechou estável, aos 51.395pts. Destaques: GOLL4
+4,04% e OIBR4 -5,98%. http://t.co/j9yUOf2ipY",
"Localiza (RENT3) autoriza pagamento de R$ 13,9 mi em juros sobre
capital próprio..."
]}

df = pd.DataFrame(corpus_dantestocks)

# acrescenta uma nova coluna com os tokens. Usei o apply para aplicar
# a função tokeniza à toda coluna 'tweet',criando a coluna 'tokens')
def tokeniza(texto):
  return nltk.word_tokenize(texto, language='portuguese')

df['tokens'] = df['tweet'].apply(tokeniza)

print('\n\nDataFrame após a tokenização:)
df
```

Tokenização com a NLTK (6/7)

- **Tokenização de todos os textos de um corpus**
  - Resultado do apply na coluna “tweet” (textos brutos) → nova coluna “tokens” com todos os textos tokenizados foi criada.

	tweet	tokens
0	“No momento PETR4 respeita o suporte de R$15,42”	['No', 'momento', 'PETR4', 'respeita', 'o', 'suporte', 'de', 'R', '$', '15,42', '.']
1	“28/04 O Ibovespa fechou estável, aos 51.395pts. Destaques: GOLL4 +4,04% e OIBR4 -5,98%. http://t.co/j9yUOf2ipY”	['28/04', 'O', 'Ibovespa', 'fechou', 'estável', ',', 'aos', '51.395pts', '.', 'Destaques', ':', 'GOLL4', '+4,04', '%', 'e', 'OIBR4', '-5,98', '%', '.', 'http', ':', '//t.co/j9yUOf2ipY']
2	“Localiza (RENT3) autoriza pagamento de R$ 13,9 mi em juros sobre capital próprio...”	['Localiza', '(', 'RENT3', ')', 'autoriza', 'pagamento', 'de', 'R', '$', '13,9', 'mi', 'em', 'juros', 'sobre', 'capital', 'próprio', '...']

Tokenização com a NLTK (7/7)

- **Tokenização customizada**
  - Para uma tokenização customizada, você deverá definir as suas próprias expressões regulares usando o módulo RegexpTokenizer.

```
# Exemplo 1: retornar apenas os tokens, sem pontuações
texto = 'O Rio de Janeiro, fevereiro e março!'

from nltk.tokenize import RegexpTokenizer
tokenizer = RegexpTokenizer(r'\w+')
tokens2 = tokenizer.tokenize(texto.lower())
print(tokens2)

# Exemplo 2: retornar apenas alguns sinais de pontuação especificados
tokenizer = RegexpTokenizer(r'[.?!,;]')
tokens3 = tokenizer.tokenize(texto.lower())
print(tokens3)
```

```
>
['o', 'rio', 'de', 'janeiro', 'fevereiro', 'e', 'março’]

[',', '!']
```

NLTK – Frequências de Palavras (1/2)

- **Frequência das palavras**
  - Com a lista de tokens, podemos obter a frequência de cada type (palavra do vocabulário) utilizando a classe FreqDist.
  - A função most_common(*n*) ordena os tokens por frequência.
    - Se *n* não for especificado, gera resultado para todos os tokens. Caso contrário, para os *n* mais frequentes.

```
texto = """Que não é o que NÃO pode ser que não é o que não pode ser
que não é o que não pode ser que não é o que NÃO PODE ser que NÃO É"""

# 1. obtém os tokens (veja que o texto foi convertido para minúsculo)
tokens = nltk.word_tokenize(texto.lower())

# 2. usa FreqDist sobre a lista de tokens em minúsculo
freq = nltk.FreqDist(tokens)

# 3. imprime os tokens, os types e e suas frequências
print('vocabulário: ', set(tokens)) # o set elimina repetições da lista
                                    # assim, gera o vocabulário!!!
print('\nfrequência de cada type (palavra):')
print(freq.most_common())
```

```
vocabulário: {'é', 'pode', 'não', 'o', 'ser', 'que'}

frequência de cada type (palavra):
[('que', 9), ('não', 9), ('é', 5), ('o', 4), ('pode', 4), ('ser', 4)]
```

NLTK – Frequências de Palavras (2/2)

- **Frequência das palavras**
  - **IMPORTANTE**: Tokens em maiúsculo e minúsculo são considerados diferentes, por isso convertemos o texto para minúsculo nos exemplos
  - Existe também a função tabulate() que gera uma tabulação, além de outros métodos disponíveis.

```
texto = """Que não é o que NÃO pode ser que não é o que não pode ser
que não é o que não pode ser que não é o que NÃO PODE ser que NÃO É"""

# 1. obtém os tokens
tokens = nltk.word_tokenize(texto.lower())

# 2. usa FreqDist sobre a lista de tokens em minúsculo
freq = nltk.FreqDist(tokens)

freq.tabulate() # gera uma tabulação (há outros métodos disponíveis)
```

que	não	é	o	pode	ser
9	9	5	4	4	4

NLTK – *n*-Gramas (1/4)

- **Bigramas, Trigramas e *n*-gramas***
  - **Sequências** de 2, 3 ou *n* **palavras consecutivas**, respectivamente.
  - Podem ser gerados a partir da lista de tokens.

```
from nltk import bigrams
from nltk import trigrams

d = 'Rio de Janeiro, São Paulo e Minas Gerais estão na região Sudeste'

# 1. gera a lista de tokens
tokens = nltk.word_tokenize(d, language='portuguese')

# 2. gera as listas com os bigramas e trigramas a partir dos tokens
lst_bigramas = list(bigrams(tokens))
lst_trigramas = list(trigrams(tokens))
print('bigramas: ', lst_bigramas); print()
print('trigramas: ', lst_trigramas)
```

```
bigramas: [('Rio', 'de'), ('de', 'Janeiro'), ('Janeiro', ','), (',', 'São'),
('São', 'Paulo'), ('Paulo', 'e'), ('e', 'Minas'), ('Minas', 'Gerais'), ('Gerais',
'estão'), ('estão', 'na'), ('na', 'região'), ('região', 'Sudeste')]

trigramas: [('Rio', 'de', 'Janeiro'), ('de', 'Janeiro', ','), ('Janeiro', ',',
'São'), (',', 'São', 'Paulo'), ('São', 'Paulo', 'e'), ('Paulo', 'e', 'Minas'),
('e', 'Minas', 'Gerais'), ('Minas', 'Gerais', 'estão'), ('Gerais', 'estão',
'na'), ('estão', 'na', 'região'), ('na', 'região', 'Sudeste')]
```

** Na primeira semana de aula vimos os n-gramas de caracteres. Mas agora estão sendo apresentados os **n-gramas de palavras** que são mais utilizados na prática e, por isso, serão chamados apenas de "n-gramas".*

NLTK – *n*-Gramas (2/4)

- **Bigramas, Trigramas e n-gramas**
  - Exemplo para identificar palavras compostas

```
# Um bigrama em que as duas palavras começam com maiúsculo
# pode vir a ser uma palavras composta
v1 = [b for b in lst_bigramas if b[0][0].isupper() and
b[1][0].isupper()]
print(v1)

# Um trigrama em que as duas palavras nas pontas começam com
# maiúsculo e a do meio com minúsculo, pode vir a ser uma palavras
# composta
v2 =[t for t in lst_trigramas if t[0][0].isupper() and
t[1][0].islower() and t[2][0].isupper()]
print(v2)
```

```
[('São', 'Paulo'), ('Minas', 'Gerais')]

[('Rio', 'de', 'Janeiro'), ('Paulo', 'e', 'Minas')]
```

NLTK – *n*-Gramas (3/4)

- **Exemplo com *4*-gramas**

```
from nltk import ngrams # para 4-gramas ou mais

texto = """Que não é o que NÃO pode ser que não é o que não pode ser
que não é o que não pode ser que não é o que NÃO PODE ser que NÃO É"""

tokens = nltk.word_tokenize(texto.lower(), language='portuguese')
print(list(ngrams(tokens, 4)))
```

```
[('que', 'não', 'é', 'o'), ('não', 'é', 'o', 'que'), ('é', 'o', 'que', 'não'),
('o', 'que', 'não', 'pode'), ('que', 'não', 'pode', 'ser'), ('não', 'pode', 'ser',
'que'), ('pode', 'ser', 'que', 'não'), ('ser', 'que', 'não', 'é'), ('que', 'não',
'é', 'o'), ('não', 'é', 'o', 'que'), ('é', 'o', 'que', 'não'), ('o', 'que', 'não',
'pode'), ('que', 'não', 'pode', 'ser'), ('não', 'pode', 'ser', 'que'), ('pode',
'ser', 'que', 'não'), ('ser', 'que', 'não', 'é'), ('que', 'não', 'é', 'o'),
('não', 'é', 'o', 'que'), ('é', 'o', 'que', 'não'), ('o', 'que', 'não', 'pode'),
('que', 'não', 'pode', 'ser'), ('não', 'pode', 'ser', 'que'), ('pode', 'ser',
'que', 'não'), ('ser', 'que', 'não', 'é'), ('que', 'não', 'é', 'o'), ('não', 'é',
'o', 'que'), ('é', 'o', 'que', 'não'), ('o', 'que', 'não', 'pode'), ('que', 'não',
'pode', 'ser'), ('não', 'pode', 'ser', 'que'), ('pode', 'ser', 'que', 'não'),
('ser', 'que', 'não', 'é')]
```

NLTK – *n*-Gramas (4/4)

- **FreqDist** – pode ser usado para contagem de frequências de bigramas, trigramas e *n*-gramas (no exemplo abaixo, contamos os bigramas)

```
from nltk import bigrams

texto = """Que não é o que NÃO pode ser que não é o que não pode ser
que não é o que não pode ser que não é o que NÃO PODE ser que NÃO É"""

tokens = nltk.word_tokenize(texto.lower(), language='portuguese')

# computando frequências de bigramas
bgs = bigrams(tokens)
fdist = nltk.FreqDist(bgs)
for k,v in fdist.items(): print(k,v)
```

```
('que', 'não') 9
('não', 'é') 5
('é', 'o') 4
('o', 'que') 4
('não', 'pode') 4
('pode', 'ser') 4
('ser', 'que') 4
```

NLTK – Normalização – Stop Words (1/5)

- **Stop words**
  - São palavras que de maneira geral **não** constituem **informação relevante** para o sentido do texto.
    - Quase sempre são palavras das seguintes classes gramaticais:
      - artigos, preposições, pronomes, conjunções, alguns verbos de ligação, ...
    - Por exemplo, a NLTK vem com uma lista de 207 stop words do Português.
      - Mas você pode incluir ou remover palavras de desejar.
      - Abaixo, são mostradas 20 das 207 stop words da lista da NLTK:

['a', 'à', 'ao', 'aos', 'aquela', 'aquelas', 'aquele', 'aqueles', 'aquilo', 'as', ..., 'tivessem', 'tivéssemos', 'tu', 'tua', 'tuas', 'um', 'uma', 'você', 'vocês', 'vos']

NLTK – Normalização – Stop Words (2/5)

- **Remoção de Stop Words**
  - Na maioria das vezes, as stop words são removidas dos textos durante a tokenização.
    - **Objetivos**: reduzir o vocabulário e melhorar processo de análise de texto.
    - Entretanto, para certas aplicações a remoção de stop words pode não ser vantajosa, tendo em vista que elas podem indicar contexto.
  - Uma lista de stop words é chamada de **stoplist**.
    - stoplist = {“à”, “a”, “cá”, “com”, “como”, “e”, “ele”, “ela”, “eu”, “dela”}
  - As ferramentas de PLN costumam vir com stoplists padrão para diferentes idiomas (como é o caso da NLTK).
    - Mas permitem que elas sejam alteradas.

NLTK – Normalização – Stop Words (3/5)

- **Como criar uma stoplist?**
  - A definição de uma stoplist é dependente de duas coisas:

    (1) **Idioma dos textos** do corpus.

    - StopList para Inglês
    - stoplist = {“a”, “and”, “in”, “I”, “he”, “she”, “these”, “this”}

    - Stoplist para Português
    - stoplist = {“um”, “e”, “no”, “eu”, “ele”, “ela”, “estes”, “este”}

    (2) **Problema** que está sendo tratado:

    - **Ex.**: Stoplist para um corpus de contratos sociais de empresas.
    - poderia ser preciso acrescentar {“ltda”, “s/a”, “me”} à stoplist.

NLTK – Normalização – Stop Words (4/5)

- **Stop Words**
  - A NLTK vem com uma lista de 207 stop words para o Português.
    - Você pode remover ou incluir qualquer termo que desejar.
  - No Colab é sempre preciso fazer o download do módulo...
  - ... mas no python da sua máquina, isso já foi resolvido no momento que você instalou a NLTK e baixou seus dados.

```
nltk.download('stopwords')
```

NLTK – Normalização – Stop Words (5/5)

- **Stop Words**
  - O exemplo abaixo ilustra como remover stop words da lista de tokens

```
# obtém a stoplist do português
stoplist = nltk.corpus.stopwords.words('portuguese')

# ---------------------------------------------------
# Exemplo: removendo as stop words de um texto
#           usando a lista padrão da nltk
# ---------------------------------------------------
stoplist = nltk.corpus.stopwords.words('portuguese')
texto = "O sorriso do cachorro está no rabo."
tokens = nltk.word_tokenize(texto.lower())
print(tokens)

tokens_sem_sw = [token for token in tokens if token not in stoplist]
print('tokens:', tokens_sem_sw)
```

```
> tokens: ['sorriso', 'cachorro', 'rabo', '.']
```

NLTK – Normalização – Lematização

- **O que é Lema?**
  - Lema é a forma de entrada de verbetes em dicionários (*definição muito simplista...*)
    - Por exemplo, você não encontra "cantei", "cantava", "cantaremos" no dicionário, mas sim "cantar" (verbo no infinitivo).
    - Da mesma forma, você não encontra "meninos" e "meninozinho" e sim "menino" (palavra no masculino singular).

- **O que é Lematização?**
  - Processo de transformar palavras para **lemas**.
  - Precisa ser feita com o **auxílio de recursos linguísticos** (alguma fonte de conhecimento, como dicionário, corpus anotado ou ontologia).
    - **Objetivos**: reduzir o vocabulário e unificar formas variantes de uma palavra para sua forma "base" (de verbete).

  - A NLTK **não possui** lematizador para o Português.
  - Porém, a spaCy tem. Veremos em uma aula futura.

NLTK – Normalização – Stemming (1/10)

- **O que é radicalização (stemming)?**
  - Processo de transformar formas variantes das palavras para uma representação concisa, denominada **stem**.
  - Essa representação concisa **não** é o **lema**!
    - Na verdade, o stemming é mais simples do que a lematização, pois não depende do emprego de um recurso linguístico.
    - O stem não precisa nem mesmo representar, necessariamente, o radical morfológico de uma palavra.

PALAVRA	STEM
acidentada	acident
acidentado	acident
acidentados	acident
acidentais	acident
acidentalmente	acident
acidente	acident
acidentes	acident

NLTK – Normalização – Stemming (2/10)

- **Stemmers**
  - Algoritmos de stemming também são chamados de **stemmers**.
  - A utilização de stemmers oferece as mesmas 2 vantagens da utilização de lematizadores:
    1. Torna o sistema mais eficiente, pois reduz a quantidade de palavras do vocabulário.
    2. Torna o sistema mais eficaz, pois unifica as formas variantes de uma palavra em uma única representação.
  - A **vantagem** do stemmer é ser **mais simples**
    - (a lematização é um processo mais complexo que depende de recurso linguístico).
  - A **desvantagem** é que ele **comete erros** que os lematizadores não cometem.
    - (understemming e overstemming, *como veremos adiante*).

NLTK – Normalização – Stemming (3/10)

- **Stemmers – prefixos e sufixos**
  - O stem de uma palavra pode ser obtido através da **remoção** de **sufixos** e/ou **prefixos** que estejam presentes nesta palavra.

Palavra	Sufixo	Prefixo	Stem
Específico	ífico	Ø	espec
Nacionais	ais	Ø	nacion
Internacional	al	inter	nacion
Escolar	ar	Ø	escol
Pré-escolar	ar	pré-	escol

  - Porém, maioria dos algoritmos de stemming trabalha **apenas** com a **remoção dos sufixos**.
    - Isso porque, a remoção de prefixos nem sempre é vantajosa, pois, muitas vezes, os prefixos mudam o significado de uma palavra.
    - **Ex.**: "aplic**ável**" e "**in**aplic**ável**".

NLTK – Normalização – Stemming (4/10)

- **Stemmers podem cometer 2 tipos de erros**:
  - **Overstemming**: quando muitas letras são removidas e, por isso, palavras que possuem significados diferentes passam a pertencer ao mesmo stem.

Palavra	Stem
Computador	comp
Computadores	comp
Comparar	comp
Comparou	comp

  - **Understemming**: quando letras demais são mantidas no stem, fazendo com que palavras que tenham o mesmo significado ou significado similar passem a pertencer a stems diferentes.

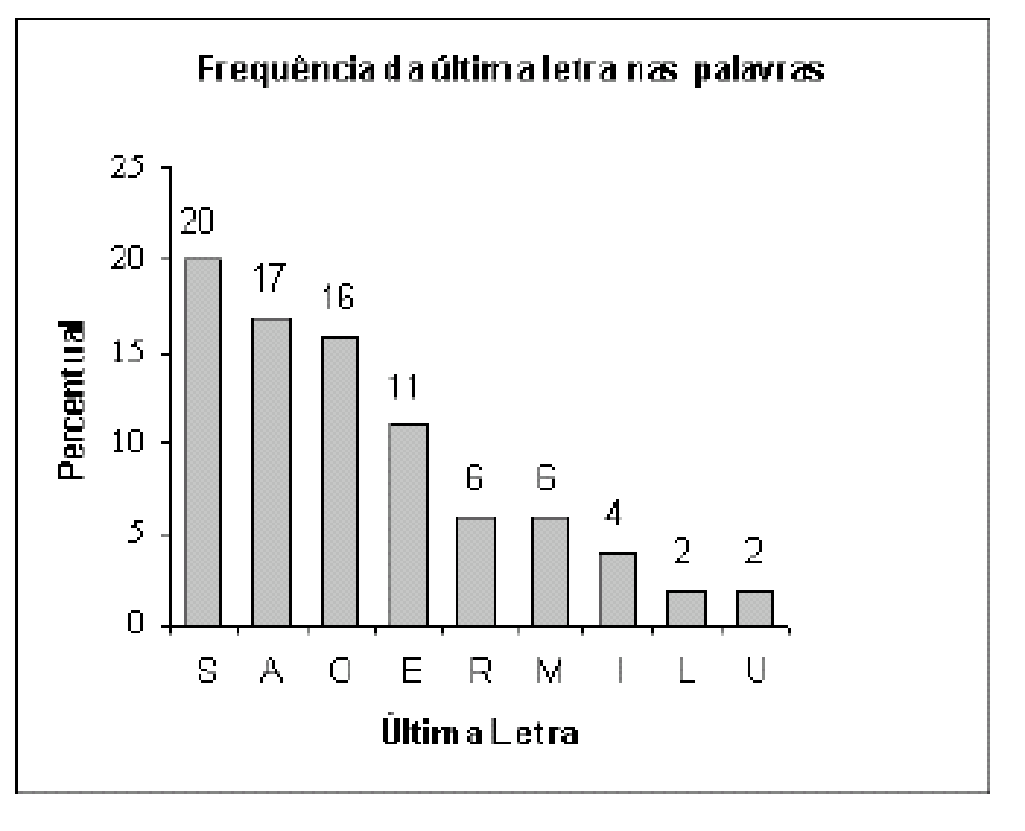
Palavra	Stem
Movimento	moviment
Movimentação	movimentaç

NLTK – Normalização – Stemming (5/10)

- **Stemmers Conhecidos**
  - O algoritmo mais tradicional para a obtenção de stems é o **Algoritmo de Porter**.
    - Este algoritmo foi projetado originalmente para a língua Inglesa, mas foi adaptado para diversas linguagens.
    - Infelizmente, ele não funciona bem para a Língua Portuguesa.
  - Por este motivo, algoritmos específicos para stemming em Português foram propostos, como:
    - **RSLP** (ORENGO; HUYCK, 2001), implementado na biblioteca NLTK.
    - **StemBR** (ÁLVARES; GARCIA; FERRAZ, 2005)

NLTK – Normalização – Stemming (6/10)

- **Algoritmo StemBR**
  - O algoritmo STEMBR é baseado num estudo estatístico realizado com 130.000 palavras em Português.
  - A partir deste estudo, foi obtida uma lista que apresenta a frequência da última letra das palavras.
  - As nove letras mais comuns da lista representam aproximadamente 85% das ocorrências.



Fonte: ÁLVARES; GARCIA; FERRAZ (2005).

IBGE
Escola Nacional de Ciências Estatísticas ENCE

NLTK – Normalização – Stemming (7/10)

- **Algoritmo StemBR**
  - O StemBR funciona de uma forma muito simples:
    - O sistema mantém 9 listas de sufixos em memória:
    - Lista de sufixos que terminam com ‘s’, ‘a’, ‘o’, ‘e’, ‘r’, ‘m’, ‘i’, ‘l’, ‘u’.
  - Para cada palavra a ser analisada, o algoritmo observa a última letra e consulta a lista de sufixos adequada.
  - A lista de sufixos é ordenada por tamanho, de maneira decrescente, para permitir a **remoção do maior sufixo** da palavra.

Palavra:
irritadíssima

→

Sufixos ‘a’
abilíssima
adíssima
alista
cava
ima
za
a

→

Stem:
irrit

NLTK – Normalização – Stemming (8/10)

- **Algoritmo StemBR**
  - Para cada sufixo, pode existir uma **lista de exceções** associada.
    - No exemplo abaixo:
      - O stem de "aceleremos" é "**aceler**" (sufixo = "emos")

```
-<sufixo>
      <descricao>iremos</descricao>
</sufixo>

-<sufixo>
      <descricao>eremos</descricao>

      <excecao>aceleremos</excecao>

  </sufixo>
<sufixo>
      <descricao>emos</descricao>
</sufixo>
```

NLTK – Normalização – Stemming (9/10)

- A NLTK possui a implementação do algoritmo RSLP, que funciona de forma muito similar ao StemBR (porém é um pouco mais simples).
  - Detalhes sobre o RSLP em:
    - https://www.inf.ufrgs.br/~viviane/rslp/index.htm
    - https://www.researchgate.net/publication/3940125_A_Stemming_Algorithmm_for_the_Portuguese_Language

- Antes de usar o stemmer RSLP no Colab é sempre preciso fazer o download do módulo...
- ... mas no python da sua máquina, só no momento da instalação da NLTK

```
nltk.download('rslp')
```

NLTK – Normalização – Stemming (10/10)

- **Exemplo:**

```
# Exemplo: uso do stemmer RSLP da NLTK
# (veja que ocorrerá understemming)
from nltk.stem import RSLPStemmer

lst = ['acidentada', 'acidentado', 'acidentados',
'acidentais', 'acidentalmente', 'acidente', 'acidentes']

stemmer = nltk.RSLPStemmer()
for palavra in lst:
  print(palavra, '--->', stemmer.stem(palavra))
```

```
acidentada ---> acident
acidentado ---> acident
acidentados ---> acident
acidentais ---> acident
acidentalmente ---> acident
acidente ---> acid
acidentes ---> acid
```

A Nossa Tabela de Técnicas !!!

- **Tabela: propriedades das técnicas apresentadas em nosso curso**

Técnica	Principais Aplicações	Dimensão	Paradigma
***n*-Gramas de Caracteres**	*features* para modelos de identif. de plágio, detecção de idioma, atribuição de autor etc.; comparação de strings.	Morfológica	Simbólico
Bag of Words (BoW)	*features* para Naive Bayes e outros algoritmos	Morfológica	Simbólico*
Naïve Bayes Multinomial	análise de sentimentos, classificação de texto	Morfológica	Estatístico
Perceptron	classificação binária de texto	Fonética a Semântica**	Estatístico, Neural
StemBR, RSLP	stemming	Morfológica	Simbólico

* A BoW para um documento é gerada de acordo com uma regra pré-definida e não depende dos demais documentos da mesma base.

** Depende das *features* utilizadas.

Referências

- ALVARES, R. V.; GARCIA, A. C. B.; FERRAZ, I. N.. STEMBR: a stemming algorithm for the brazilian portuguese language. *In*: Portuguese Conf. on Artificial Intelligence, 12., 2005, Covilha. **Anais** [...] Heidelberg : Springer Berlin, 2005. p. 693 - 701.

- FINATTO, M. J. B. et al. Sequência de Caracteres e Palavras. In: CASELI, H. M.; NUNES, M. G. V. (org.). **Processamento de Linguagem Natural: Conceitos, Técnicas e Aplicações em Português**. 3. ed. [s.l.] : BPLN, 2024. Disponível em: https://brasileiraspln.com/livro-pln/3a-edicao/parte-palavras/cap-caracteres-palavras/cap-caracteres-palavras.html. Acesso em: 11 jan. 2025.

- FREITAS, C. Dataset e Corpus. In: CASELI, H. M.; NUNES, M. G. V. (org.). **Processamento de Linguagem Natural: Conceitos, Técnicas e Aplicações em Português**. 3. ed. [s.l.] : BPLN, 2024. Disponível em: https://brasileiraspln.com/livro-pln/3a-edicao/parte-dados-avaliacao/cap-dataset-corpus/cap-dataset-corpus.html. Acesso em: 27 mar. 2025.

- JURAFSKY, D.; MARTIN, J. H. Regular Expressions, Text Normalization, Edit Distance. *In*: JURAFSKY, D.; MARTIN, J. H. **Speech and Language Processing**. 3. ed. (Jan. 6, 2026 draft). [*s.l.*]: [*s.n.*], 2026. cap 2, p. 1 – 25. Disponível em: < https://web.stanford.edu/~jurafsky/slp3/ >. Acesso em: 18 mar. 2026.

- ORENGO, V.; HUYCK, C. A Stemming algorithmm for the portuguese language", *In*: String Processing and Information Retrieval, International Symposium, 2001, Laguna de San Rafael. **Anais** [...] [*s.l.*] : IEEE, 2001. p. 186 - 193.

- PARDO, T. A. S.; SANTOS, R. L. S.; SOUSA, R. F. Python para Processamento de Linguagem Natural: Edição Comemorativa dos 50 anos do ICMC. **ICMC-USP**. São Carlos, 01 set 2021. Disponível em: https://cursosextensao.usp.br/course/view.php?id=2721. Acesso em: 05 jan. 2024.